

Git & GitHub — Reinforcement Practice Paper (Roman Urdu)

Instructions: Pehle khud se saare questions solve karo bina notes dekhe, phir end mein diye gaye **Answer Key** se check karo. Jo galat ho jaye, wapas us concept ko notes mein revise karo.

Section A: MCQs (1 Mark Each) — 15 Questions

Q1. Git basically kya hota hai? a) Ek website b) Ek Version Control System c) Ek programming language d) Ek database

Q2. GitHub kya hai? a) Ek software jo computer pe install hota hai b) Ek version control system c) Ek website jahan developers apna code store karte hain d) Ek code editor

Q3. Kisi file ka "screenshot" lene (record karne) ko Git mein kya kehte hain? a) Add b) Push c) Commit d) Clone

Q4. GitHub pe project folders ko kya kehte hain? a) Directory b) Repository c) Branch d) Container

Q5. `.git` folder kis type ki file hoti hai? a) Normal file b) Hidden file c) README file d) Config file

Q6. Naya local Git repo banane ke liye konsi command use hoti hai? a) `git clone` b) `git init` c) `git start` d) `git new`

Q7. Kisi remote repo ko local machine pe copy lane ke liye konsi command use hoti hai? a) `git init` b) `git pull` c) `git clone` d) `git fetch`

Q8. Ek file jo kabhi bhi commit nahi hui aur Git ko uske baare mein pata nahi, uska status kya hoga? a) Modified b) Staged c) Untracked d) Committed

Q9. `git add .` command kya karti hai? a) Sirf ek file ko stage karti hai b) Saari changed/new files ko stage karti hai c) Saari files ko delete karti hai d) Repo ko clone karti hai

Q10. Default branch ka naya (current) naam kya hai (jo pehle "master" hota tha)? a) main b) master c) trunk d) default

Q11. `-u` flag `git push -u origin main` mein kya karta hai? a) File delete karta hai b) Upstream set karta hai c) Branch delete karta hai d) Commit revert karta hai

Q12. Kisi doosri branch pe switch karne ke liye konsi command use hoti hai? a) `git switch-branch` b) `git checkout branch-name` c) `git move branch-name` d) `git branch branch-name`

Q13. Merge Conflict kab hota hai? a) Jab do alag files mein changes ho b) Jab ek hi file ki same line pe do branches mein alag changes ho c) Jab commit message empty ho d) Jab branch delete ki jaye

Q14. `HEAD~1` ka matlab kya hai? a) 1 commit aage jaana b) Latest commit ko delete karna c) 1 commit peeche jaana d) Saari history delete karna

Q15. Kisi doosre ke GitHub project ki apni copy banane ko kya kehte hain? a) Clone b) Fork c) Branch d) Merge

Section B: Fill in the Blanks (1 Mark Each) — 10 Questions

1. Git ke andar changes ko finalize karne ke process ko _____ kehte hain.
2. GitHub pe repo ke andar project ki detail batane wali special file ko _____ kehte hain.
3. Jab hum kisi remote repo se local mein content download karte hain to _____ command use hoti hai.
4. `git status` mein agar koi file **stage** ho chuki ho lekin commit nahi hui to uska color VS Code mein _____ dikhta hai (hint: color name).

5. Remote GitHub repo ka default naam _____ hota hai.
 6. Do branches ke differences dekhne ke liye _____ command use hoti hai.
 7. GitHub pe do branches ko merge karne ka doosra (non-command-line) tareeka _____ kehlata hai.
 8. Har commit ka apna unique code hota hai jise _____ kehte hain.
 9. Sabse latest commit ko Git mein _____ kehte hain.
 10. Kisi commit ko poori tarah (Git + working files dono se) undo karne ke liye `git reset` ke saath _____ flag use hota hai.
-

Section C: True / False (1 Mark Each) — 8 Questions

1. Git ek website hai jo cloud pe code store karti hai. (T/F)
 2. `git commit` se pehle hamesha `git add` karna zaroori hota hai (agar change naya/modified ho). (T/F)
 3. `git branch -d` se hum us branch ko delete kar sakte hain jis pe hum khade hain. (T/F)
 4. Private repo sirf owner ko dikhti hai, public repo sabko dikhti hai. (T/F)
 5. `git push -u origin main` ke baad, future pushes ke liye sirf `git push` likhna kaafi hai. (T/F)
 6. Fork karne ke baad changes automatically original repo mein bhi ho jate hain. (T/F)
 7. `.git` folder normal `ls` command se dikhta hai (bina `-a` flag ke). (T/F)
 8. Merge conflict resolve karne ke baad bhi hume `git add` aur `git commit` karna padta hai. (T/F)
-

Section D: Command Writing (2 Marks Each) — 10 Questions

Har scenario ke liye sahi Git command likho:

1. Apna Git username globally set karna hai "Ali Shah" ke naam se.
 2. Ek repo ko `https://github.com/user/repo.git` se clone karna hai.
 3. `app.js` naam ki file ko staging area mein add karna hai.
 4. Ek commit banani hai message "Fix login bug" ke saath.
 5. `feature-login` naam ki nayi branch banani hai aur usme switch bhi karna hai.
 6. Current branch ka naam check karna hai.
 7. `feature-login` branch ko `origin` pe push karna hai.
 8. `main` branch ko `feature-login` branch ke saath merge karna hai (current branch main hai).
 9. Remote repo se latest changes local `main` branch mein lane hain.
 10. Last commit ko poori tarah undo karna hai (working files se bhi).
-

Section E: Scenario-Based Questions (3 Marks Each) — 7 Questions

Q1. Aap ne apni `feature-payment` branch pe kaam kiya, `index.js` file mein ek naya function add kiya. Aapne `git status` run kiya to file "Untracked" show ho rahi hai. Batao step-by-step ke aap is change ko GitHub tak kaise pahunchayenge (poora workflow likho).

Q2. Aapne galti se ek important file delete kar di aur ye change **commit** bhi ho chuka hai. Aapko wapas us commit se pehle wali state mein jaana hai. Konsi command use karoge aur kyun?

Q3. Do developers (A aur B) ek hi file ki same line 10 mein alag-alag changes karte hain, dono apni branches ko `main` ke saath merge karne ki koshish karte hain. Kya hoga? Isko resolve karne ka process explain karo.

Q4. Aapke paas ek naya project idea hai jo aapne apne laptop pe already start kar rakha hai (koi GitHub repo nahi banayi abhi tak). Ab aap chahte ho ye GitHub pe bhi ho. Poora process batao (`git init`) se lekar push tak.

Q5. Aap ek open-source project (jaise Express.js) mein contribute karna chahte ho lekin aapko us repo pe direct write access nahi hai. Aap kya approach use karoge? Steps likho.

Q6. Aapne 3 alag commits kiye (`Add button`), (`Add dropdown`), (`Fix bug`) lekin ab aap chahte ho sirf pehle commit (`Add button`) tak wapas jaana hai, baaki dono commits completely hatane hain. Kya karoge?

Q7. Aapki team mein ek senior developer hai jo (`main`) branch ko directly change nahi hone deta — sab log pehle apni branch pe kaam karte hain phir request bhejte hain. Ye konsa GitHub feature/process hai? Uska poora flow (create se merge tak) explain karo.

Section F: Conceptual Short Answer (2 Marks Each) — 5 Questions

1. (`git add`) aur (`git commit`) mein kya farq hai? Real-life analogy ke saath explain karo.
 2. Branch banane ka fayda kya hota hai jab multiple developers ek project pe kaam kar rahe hon?
 3. "Local" aur "Remote" repo mein kya farq hai?
 4. Clone aur Fork mein kya farq hai?
 5. (`git reset`) aur (`git reset --hard`) mein kya farq hai?
-

Answer Key

Section A (MCQs)

1. b | 2. c | 3. c | 4. b | 5. b | 6. b | 7. c | 8. c | 9. b | 10. a | 11. b | 12. b | 13. b | 14. c | 15. b

Section B (Fill in the Blanks)

1. Commit
2. README (README.md)
3. `git pull`
4. Green
5. origin
6. `git diff`
7. Pull Request (PR)
8. Hash
9. HEAD
10. `--hard`

Section C (True/False)

1. False (Git = tool/software; GitHub = website)
2. True
3. False (jis branch pe khade ho wo delete nahi ho sakti, pehle switch karna padta hai)
4. True
5. True
6. False (fork sirf apni copy hoti hai, original mein automatically kuch nahi hota — PR bhejna padta hai)
7. False (`ls -a` chahiye hidden files ke liye)
8. True

Section D (Command Writing)

1. `git config --global user.name "Ali Shah"`
2. `git clone https://github.com/user/repo.git`

3. `git add app.js`
4. `git commit -m "Fix login bug"`
5. `git checkout -b feature-login`
6. `git branch`
7. `git push origin feature-login`
8. `git merge feature-login`
9. `git pull origin main`
10. `git reset --hard HEAD~1`

Section E (Scenario-Based) — Sample Expected Answers

1. `git add index.js` → `git commit -m "message"` → `git push origin feature-payment` (agar branch pehli baar push ho rahi hai to `-u` bhi use karo).
2. `git reset HEAD~1` (agar sirf ek commit peeche jaana hai) — ye commit ko undo karega. Agar working files se bhi hatana ho to `--hard` add karo.
3. Merge Conflict hoga. Resolve karne ke liye: conflict markers dekho, decide karo current/incoming/dono changes chahiye, extra marker lines hatao, save karo, `git add` + `git commit` karo, phir push karo.
4. `git init` → GitHub pe naya repo banao (bina README ke) → `git remote add origin <link>` → `git remote -v` (verify) → `git add .` → `git commit -m "Initial commit"` → `git push -u origin main`.
5. Repo ko **Fork** karo (apne account mein copy ban jayegi) → apni fork pe changes karo → phir original repo ko **Pull Request** bhejo.
6. `git log` se `Add button` commit ka hash copy karo → `git reset --hard <hash>`.
7. Ye **Pull Request** process hai: Developer apni branch banata hai → changes commit/push karta hai → GitHub pe "Compare & pull request" click karta hai → PR create hoti hai → Senior/reviewer PR review karta hai (comment/approve) → "Merge pull request" → "Confirm merge" → PR close ho jati hai aur changes main mein aa jate hain.

Section F (Conceptual)

1. `git add` = staging (engagement jaisa) — change ko commit ke liye ready karna.
`git commit` = wedding jaisa — change ko permanently record karna.
 2. Har developer apni alag branch pe kaam karta hai, kisi aur ka wait nahi karna padta, aur baad mein sab apna kaam main branch ke saath merge kar dete hain — isse conflicts/overwrites bhi kam hote hain.
 3. Local = apna laptop/computer ka repo. Remote = GitHub pe hosted repo.
 4. Clone = ek existing repo (apni ya kisi aur ki) ko local machine pe download karna.
Fork = kisi doosre ke GitHub account ki repo ki apni khud ki GitHub copy banana.
 5. `git reset` sirf staging area/commit history ko change karta hai (working files untouched reh sakti hain depending on flag). `git reset --hard` working directory ki files ko bhi us commit ki state mein wapas le jata hai — sab uncommitted changes permanently lose ho jate hain.
-

Total Marks: 15 + 10 + 8 + 20 + 21 + 10 = **84 Marks**

Score guide:

- 70+ → Concept clear hai, revision ki zarurat nahi
- 50-70 → Weak areas identify karo aur unhi sections ko dobara padho
- <50 → Poore notes se dobara guzro phir paper dobara solve karo